Open in app ↗

Karpathy Left His GPU Running Overnight. The Agent Found a Bug Everyone Missed for Months.

“The goal is to engineer your agents to make the fastest research progress indefinitely and without any of your own involvement.” — Andrej Karpathy, March 2026. My first reaction was: that sounds like a job posting for a system that replaces me.

Then I actually ran it.

I Have Wasted a Lot of GPU Hours Doing What an Agent Can Do Better

I have run hundreds of ML experiments the manual way. Edit a file. Kick off a run. Wait. Check the loss curve. Feel vaguely disappointed. Edit a slightly different file. Repeat until midnight. If you have trained a language model, even a small one, you know what this looks like. It looks like most of your week going to babysitting a process that does not need you there.

When Karpathy published autoresearch on March 7, 2026, I read the README in about six minutes. Three files. One metric. One GPU. Fixed 5-minute time budget per experiment. The agent runs. You sleep. I thought it was a toy.

Then I saw that his own two-day run produced 700 autonomous experiments, found 20 genuine improvements on code he had already spent months hand-tuning, and dropped the Time to GPT-2 benchmark from 2.02 hours to 1.80 — an 11% gain on a project he thought was already fully optimized [1]. One of those improvements was a bug in his attention scaling he had missed entirely. The agent caught it. Not a colleague. Not a code review. The loop.

> **TL;DR:** ML experimentation is bottlenecked by human patience, not GPU compute. autoresearch hands the repetitive loop to an AI agent with a single scoreable metric, and the agent finds improvements faster than a human can think to try them. The pattern is not limited to language models. It works on anything you can score objectively.

The Dominant Approach Has a Ceiling

The standard way to tune a model is what I would call the informed guess loop. You have a hypothesis — "maybe a lower learning rate warmup would help" — you implement it, run it, check val loss, form another hypothesis. A good researcher can get through eight to ten of these cycles in a full day, and most of that time is waiting for the GPU, not thinking.

That ceiling is not a skill problem. It is a throughput problem.

Think of it like this: the best chess player in 1996 could not beat a machine that simply evaluated 200 million positions per second. The player was not dumb. The player just could not think fast enough. Manual ML experimentation has the same structural ceiling — the hypotheses are only as fast as the human forming them.

autoresearch removes that ceiling. The agent runs 12 experiments per hour. An overnight session covers roughly 100. A two-day run, like Karpathy's, covers 700. None of those experiments need a human decision between them.

The Three Pieces That Make It Actually Work

I want to spend time here because most of the coverage of autoresearch describes *what* it does and misses *why* it works. The insight is not "AI can write training code." The insight is the design constraints that make the loop reliable.

One editable file. The agent can only modify `train.py`. It cannot touch `prepare.py`, which handles data loading and evaluation. It cannot change the metric. The attack surface is intentionally narrow. This is why the loop doesn't collapse into chaos. The agent can rewrite the attention mechanism, change the optimizer, restructure the training loop — but every change is a readable diff and every failure is contained. You can review exactly what the agent tried.

One vocabulary-size-independent metric. val_bpb — validation bits per byte — measures how many bits the model needs to encode one byte of validation text, lower being better. The critical word is "byte." Most validation loss metrics are token-level, which means changing the tokenizer changes the loss scale. The agent could "improve" performance by choosing a different vocabulary size rather than building a better model. val_bpb removes that escape hatch. The agent can change tokenization, architecture, sequence length — and every experiment still gets a fair comparison on the same scale [2].

Fixed time budget, not fixed epoch count. Every training run lasts exactly five wall-clock minutes, regardless of what the agent changed. This means a larger model and a smaller model are compared on equal footing — what the hardware can actually do in that window. The consequence is that the agent finds the optimal configuration for *your specific GPU*, not an abstract optimum. Shopify CEO Tobi Lutke ran 37 experiments overnight and woke up to a 0.8B parameter model outperforming his hand-tuned 1.6B model [3]. Half the parameters. Better results. The fixed budget forced the comparison that revealed it.

```
# What the agent reads at the top of train.py
# This is the ratchet — every improvement becomes the new floor
# Failures get wiped with: git reset --hard HEAD

if val_bpb < best_val_bpb:
    best_val_bpb = val_bpb
    save_checkpoint(model, optimizer, step, val_bpb)
    # git commit happens here — this version is now the baseline
else:
    # git reset --hard HEAD — rolled back, nothing persisted
    pass
```

The git rollback is not a detail. It is the whole mechanism. Every experiment either commits and raises the floor, or reverts and leaves no trace. The name “ratchet loop” comes from exactly this: it can only move forward.

What the Agent Actually Finds (And What It Does Not)

The results from Karpathy’s extended runs and community reproductions are now well-documented. The agent finds structural code changes — missing scaler multipliers in QKNorm, beneficial value embedding

regularization, better AdamW beta parameters, tighter gradient clipping — the kind of thing a methodical engineer would eventually find but would take days to isolate [4].

Red Hat ran autoresearch on OpenShift AI with H100 GPUs for 24 hours straight. 198 experiments, 29 kept, 2.3% improvement in validation loss, zero human intervention required [5]. The staircase improvement pattern Karpathy described held up across independent infrastructure.

What the agent does not do: invent novel architectures, reason across multiple experiments simultaneously, or generalize from one domain to an adjacent one. Each experiment is evaluated on its own. A change that sets up a better change two steps later will get rolled back if it doesn't immediately improve val_bpb. The ratchet loop is greedy by design. That is also its constraint.

Where This Breaks, and Why It Matters to Know

autoresearch works when you have a single, objective, computable metric. That condition is harder to satisfy than it looks.

For classification tasks, accuracy is usually fine. For regression, MSE or RMSE works. For language model pretraining, val_bpb is ideal precisely because it handles vocabulary changes. These are the clean cases.

The pattern fails when the metric can be gamed without actually improving the model. If you set val_bpb on a too-easy validation set, the agent will overfit to it. If your eval function is slow to compute, the 5-minute budget fills up with evaluation rather than training. If your metric has

discontinuities — accuracy on a small dataset jumping around due to randomness — the agent will chase noise.

It also fails on multi-objective problems. If you need to balance accuracy and latency, you need a composite metric that honestly reflects both. The agent will optimize whatever you hand it. Handing it the wrong thing produces confident, automated mediocrity.

And for very large models, the 5-minute budget is simply not enough to see meaningful signal. Karpathy explicitly designed this for single-GPU, small-to-medium model training. It is not a distributed training harness. Trying to use it as one will get you 100 experiments that all show noise.

The correct framing: autoresearch is a research tool, not a deployment pipeline. Use it to find a good configuration. Build the production pipeline separately, with the findings in hand.

Try It This Weekend — Three Entry Points

If you want to run it yourself, the barrier is lower than the coverage suggests.

Start with the smallest possible step: clone the repo, run `uv run prepare.py`, and do a single manual `uv run train.py`. That is five minutes. You will see val_bpb computed, a checkpoint saved, and the git diff that would have been committed if the agent had been running. Once that works, you have confirmed your setup.

If that runs cleanly, open `program.md` and write two or three sentences describing what you want explored — something like "try reducing the number of attention heads and compensating with a wider hidden

dimension." Then start the loop with Claude Code or any coding agent pointed at the repo. Leave it for an hour and check the experiment log when you come back.

If you want to go further, the community forks at the bottom of the GitHub README cover RTX cards, Apple Silicon M1-M4, and AMD GPUs. There are also forks applying the pattern to domains outside language modeling, including query optimization and compiler flag tuning [6].

The improvement that changes how you think about the problem usually shows up around experiment 15 or 20. Not because that number is magic — but because by then the agent has exhausted the obvious changes and started finding the ones you would not have thought to try.

That is when it gets interesting.

References

[1] Karpathy, A. (2026, March). *autoresearch — AI agents running research on single-GPU nanochat training automatically.* GitHub. https://github.com/karpathy/autoresearch

[2] VentureBeat. (2026, March 10). *Andrej Karpathy's new open source 'autoresearch' lets you run hundreds of AI experiments a night.* https://venturebeat.com/technology/andrej-karpathys-new-open-source-autoresearch-lets-you-run-hundreds-of-ai

[3] Product Growth Newsletter. (2026, March 20). *Autoresearch Guide: The 42K-star repo everyone thinks is for ML researchers.* https://www.news.aakashg.com/p/autoresearch-guide-for-pms

[4] Verdent AI Guides. (2026). *AutoResearch Explained: How Karpathy's AI Research Agent Works.* https://www.verdent.ai/guides/what-is-autoresearch-karpathy

[5] Red Hat Developer. (2026, April 7). *Running Karpathy's autoresearch on Red Hat OpenShift AI: 198 experiments, zero intervention.* https://developers.redhat.com/articles/2026/04/07/autoresearch-on-red-hat-openshift-ai-198-experiments-zero-intervention

[6] DataCamp. (2026, March 23). *A Guide to Andrej Karpathy's AutoResearch: Automating ML with AI Agents.* https://www.datacamp.com/tutorial/guide-to-autoresearch